
The Explore → Plan → Code → Verify Cycle

Week 4 — Disciplined Vibe Coding

From vibes to discipline — the workflow that separates hobbyists from engineers



Instructor: Goker Ezberci

Vibe Coding 101: From Idea to Shipped Product

ABL Poll — Always Be Learning

1 How many of you tried adding a new feature after Week 3?

2 Did the AI “forget” your project’s patterns?

3 Who broke something that was working?

If you answered yes to #2 or #3 — today’s lecture solves that.

Where We Are — Weeks 1-3 Recap

Week 1

From Zero to Vibe Coding

Setup + first AI conversation

Dev environment, first AI-generated code

Week 2

PRD & Planning

PRD-First Development

Your PRD with Non-Goals & Agent Boundaries

Week 3

Ship It Week

Deployment & CI/CD

Deployed prototype with public URL

Today: You have a working prototype. Now we make it ***maintainable.***

Your Project Structure — GitHub

What your repo should look like by the end of today

```
your-project/
├── CLAUDE.md           ← NEW THIS WEEK
├── .cursorrules        ← Same content
├── GEMINI.md           ← Same content
├── package.json
├── docs/
│   └── PRD.md          ← From Week 2
├── src/
│   ├── api/routes/     Route handlers
│   ├── services/       Business logic
│   ├── models/         DB schema
│   ├── middleware/     Auth, errors
│   └── __tests__/       Mirrors src/
├── .github/
│   └── workflows/      ← CI/CD Week 3
└── .gitignore
```

Instruction Files

CLAUDE.md, .cursorrules, GEMINI.md — your AI’s “first day briefing.” Same content, different filenames per tool.

PRD from Week 2

Your product requirements document with Non-Goals and Agent Boundaries. AI reads this for project context.

Tests & CI/CD

__tests__ / mirrors your src/ structure. GitHub Actions run tests on every push.

Don’t have this yet? That’s OK. **Today’s lab will get you there.**

Why We Use CLI, Not IDE

	CLI (Gemini CLI, Claude Code)	IDE (Cursor, Windsurf, Copilot)
Transparency	You see every prompt & response	Magic diffs — click “Accept” blindly
Context	You control what AI reads	IDE picks files silently
Learning	Forces you to think about context	Abstracts away the skill
Cost	Free (Gemini CLI)	\$20/month subscriptions
Transferable	Skills work in any tool	Locked to one IDE
Composable	Chains with git, npm, CI/CD	Walled garden

Learn CLI first → pick up any IDE in 30 minutes. The reverse doesn't work.

We're not anti-IDE. But learning the discipline first means you'll use them better. Like learning manual before automatic.

Learning Objectives — Week 4

Apply

the EPCV cycle to add features to existing codebases

Create

instruction files (CLAUDE.md, .cursorrules, GEMINI.md) that provide persistent AI context

Implement

a disciplined git workflow for AI-assisted development with atomic commits

Evaluate

AI-generated code against quality criteria before committing

The Problem — Why Your Prototype Is Fragile

What happens when you add features to Week 3's prototype:

You: "Add a search feature"

AI generates code that...

- x Ignores your existing API patterns
- x Uses a different CSS framework
- x Duplicates a utility you already have
- x Breaks the navigation component



WHY?

The AI doesn't know your codebase.

Every new conversation starts from zero. No memory of your patterns, architecture, or decisions.

The 70% Wall

Weeks 1-3: Get to 70% fast with AI

Weeks 4-12: Push past the wall with **DISCIPLINE**

✓ DONE

◀ **YOU ARE HERE**

Today's solution: The Explore → Plan → Code → Verify cycle

Part 1 — The EPCV Cycle

The disciplined workflow for AI-assisted development



EXPLORE

5 min

Understand before changing

Read code, check patterns, review git history



PLAN

5 min

Design before building

Get AI plan — no code yet!



CODE

15 min

Implement step by step

One step at a time, review each step



VERIFY

5 min

Test, review, commit

Tests + diff review + atomic commit

Golden rule: Never skip EXPLORE. It prevents 80% of bugs.

🔍 EXPLORE — Understand Before Changing

Gather intelligence. Don't modify anything.

WHAT TO EXPLORE

- File structure & architecture patterns
- Existing tests and coverage
- Recent git history for the area
- Dependencies & integration points
- Naming conventions & code style

```
# Gemini CLI (FREE)
gemini "Read src/auth/ directory.
Explain the auth flow.
Don't modify anything."
```

```
# Claude Code
claude "Explore the project.
What patterns does this use?"
```

🛡️ What EXPLORE Prevents

- ✗ AI generating code that duplicates existing functionality
- ✗ Breaking patterns the codebase already follows
- ✗ Missing integration points that cause runtime bugs
- ✗ Reinventing utilities that already exist in your project

Key: Tell the AI “don't modify anything.” Without this, it starts writing code immediately.

PLAN — Design Before Building

Get a plan. Verify it makes sense. THEN code.

A GOOD PLAN INCLUDES

- 1 Which files need to change
- 2 What new files are needed
- 3 Edge cases to handle
- 4 Tests to write
- 5 Potential breaking changes

```
# Gemini CLI (FREE)
gemini "Plan how to add search."
```

```
List:
  1. Files to change
  2. Edge cases
  3. Tests to write
```

```
Don't write code yet
— just the plan."
```

What PLAN Prevents

Over-engineering (AI loves adding unrequested features) • **Architectural mistakes** (expensive to fix later) • **Missing edge cases** (production bugs) • **Scope creep** (the plan becomes your contract)

Critical instruction: “Don’t write code yet — just the plan.” This is the most powerful AI directive.

</> CODE — Implement Step by Step

Execute one step at a time from the plan. Review each before proceeding.

⚠ BAD — ALL AT ONCE

```
"Implement the entire search feature"
```

- ✗ 500-line diff, impossible to review
- ✗ Bugs hidden in massive changes
- ✗ Can't revert one thing without losing all
- ✗ AI quality drops with unfocused context

✅ GOOD — STEP BY STEP

```
"Step 1: Create search API"  
  → Review ✓  
"Step 2: Add search UI component"  
  → Review ✓  
"Step 3: Connect UI to API"  
  → Review ✓ Done!
```

- ✓ Small diffs, easy to review
- ✓ Know exactly which step broke
- ✓ Revert one step, keep the rest



The temptation is to let AI do everything at once. **Resist.** Small steps = fewer bugs. AI quality is **higher** with focused context.

✓ VERIFY — Test, Review, Commit

Nothing ships without verification. The AI is always confident — your job is to verify.

1 `npm test` Nothing broke

2 `Write new tests` New code is covered

3 `git diff` No surprises in the diff

4 `git add src/search/...` Stage specific files only

5 `git commit -m "feat: ..."` Clear commit history

NEVER: `git add .` — stages everything, including files you didn't mean to commit

ALWAYS: `git add [specific files]` — you control what ships

EPCV Cycle — Complete Example

Adding "password reset" using the full cycle

🔍 EXPLORE

- Read auth module
- Check token patterns
- Review email setup



📋 PLAN

- 1. Reset-request endpoint
- 2. Token generation
- 3. Email integration
- 4. Reset-confirm
- 5. Write 4 tests



</> CODE

- Step 1 → Review ✓
- Step 2 → Review ✓
- Step 3 → Review ✓
- Step 4 → Review ✓



✅ VERIFY

- npm test (all pass)
- 4 new tests added
- git diff reviewed
- Atomic commit ✓

TIME COMPARISON

"Just vibe it"

30 min

3–5 bugs in production

EPCV cycle

45 min

0–1 bugs (caught in VERIFY)

15 extra minutes saves 3+ hours of debugging.

Anti-Patterns — What NOT to Do

The YOLO Pattern

"Add all features at once" → 1000-line diff, impossible to review

The Copy-Paste Pattern

Copy AI output → Paste → Ship. No understanding of what changed.

The Infinite Conversation

50 messages about 5 different features → Context polluted, AI confused

The "AI Said It's Fine"

"Is this correct?" → AI: "Yes!" — AI is always confident. Test it yourself.

The fix for all of these: **EPCV. Every feature. Every time.**

Part 2 — Instruction Files

Context engineering at the project level — persistent AI context that travels with your code

FILE	TOOL	LOCATION
CLAUDE.md	Claude Code	Project root
.cursorrules	Cursor	Project root
GEMINI.md	Gemini CLI	Project root
.github/copilot-instructions.md	GitHub Copilot	.github/ folder
.windsurfrules	Windsurf	Project root

💡 Key Insight

These files use the **same principles**. Learn one, adapt to any. The content is 90% the same across tools — only the format changes slightly.

Instruction Files — What, Not What, Where

✓ WHAT THEY ARE

- ✓ A markdown file in your project root
- ✓ Persistent context that auto-loads every conversation
- ✓ Your project's "brain" — rules, patterns, decisions
- ✓ Version-controlled (lives in git, travels with code)
- ✓ The bridge between your knowledge and AI's output

⚠ WHAT THEY ARE NOT

- ✗ Not a prompt (auto-loads — you don't paste it)
- ✗ Not documentation (written FOR AI, not humans)
- ✗ Not a tutorial (no explanations — just rules & facts)
- ✗ Not optional (without it, every chat starts from zero)
- ✗ Not a one-time thing (evolves with your project)

≡ WHERE THEY GO

```
your-project/  
├─ CLAUDE.md  
├─ GEMINI.md  
├─ .cursorrules  
├─ .windsurfrules  
├─ .github/  
│   └─ copilot-  
│       instructions.md  
  
src/auth/  
└─ CLAUDE.md  
   (scoped rules)
```

One file. Project root. Auto-loaded. The AI reads it before every conversation — like a dev reading the README on their first day.

Scoped files (e.g. src/auth/CLAUDE.md) override root rules for specific dirs. Start with one root file.

The 6 Sections Every Instruction File Needs

“Context engineering is the delicate art of filling the context window with just the right information.” — Andrej Karpathy, 2025

1 COMMANDS

How to build, test, deploy

2 STRUCTURE

Where code lives

3 DECISIONS

Architecture already chosen

4 STYLE

Code conventions

5 TESTING

Test framework & rules

6 BOUNDARIES

Always / Never / Ask First

💡 Order Matters

Commands at top, Boundaries at bottom — where AI attention peaks.

Faros AI 5-Layer Context Strategy

🛡️ Keep It Short

<60 lines (HumanLayer) to <300 lines (Anthropic).
Shorter files = higher AI compliance.

Anthropic Best Practices (2026)

Sources: [Anthropic Best Practices \(2026\)](#) · [MSR '26 "Beyond the Prompt"](#) · [Martin Fowler / Thoughtworks](#) · [Faros AI 5-Layer Strategy](#) · [Karpathy on Context Engineering](#)

A Real CLAUDE.md — TaskFlow Project

58 lines. Under the 60-line sweet spot. Every line earns its place.

Commands

```
npm run dev      # Start dev server (port 3000)
npm test         # Run Vitest test suite
npm run build    # Production build
npm run lint     # ESLint + Prettier check
npm run db:migrate # Run Prisma migrations
```

Structure

```
src/
├─ api/routes/    # Route handlers (one file per resource)
├─ services/      # Business logic (no HTTP here)
├─ models/        # Prisma schema + generated types
├─ middleware/    # Auth, validation, error handling
└─ __tests__/     # Mirrors src/ structure exactly
```

Decisions

Express.js + TypeScript (strict mode)
PostgreSQL via Prisma ORM
JWT tokens in httpOnly cookies
Zod schemas for all inputs

Style

Named exports only (no default exports)
Async/await everywhere (no .then() chains)
Early returns for guard clauses
Max function length: 30 lines
File naming: kebab-case (user-service.ts)

Testing

Framework: Vitest | Coverage: 80% minimum
Location: __tests__/ mirroring src/
Pattern: arrange → act → assert (AAA)
Mock external services, never mock own code

Boundaries

ALWAYS: Run tests, Use Prisma, Validate w/ Zod
NEVER: Hardcode secrets, Use any, Skip errors
ASK: New deps, Schema changes, Auth changes

What Each Section Does for the AI

Without → With: the difference one instruction file makes

SECTION	✗ WITHOUT instruction file	✓ WITH instruction file
① COMMANDS	AI guesses npm start (wrong), can't run tests	AI runs npm test before every commit, catches bugs
② STRUCTURE	AI creates /helpers/auth.js (wrong location, .js)	AI creates /src/middleware/auth.ts (correct path)
③ DECISIONS	AI suggests MongoDB, writes raw SQL queries	AI uses Prisma ORM, matches your JWT pattern
④ STYLE	export default class + .then() chains	export const createUser = async () + early returns
⑤ TESTING	Jest tests (wrong framework), random locations	Vitest in __tests__/, AAA pattern, 80% coverage
⑥ BOUNDARIES	Hardcodes DB URL, adds 3 deps, skips errors	Asks before deps, never hardcodes, handles errors
AI cannot infer boundaries from omission. What you don't say, AI will invent.		

Boundaries — The Most Important Section

Three categories. AI cannot infer boundaries from omission — what you don't say, AI will invent.

✓ ALWAYS

Non-negotiable rules

- ✓ Run npm test before committing
- ✓ Use Prisma for all database queries
- ✓ Validate input with Zod schemas
- ✓ Log errors with structured JSON

⚠ NEVER

Hard stops

- ✗ Hardcode API keys
- ✗ Use `any` type
- ✗ Write SQL directly (use Prisma)
- ✗ Skip error handling
- ✗ Commit without tests

💡 ASK FIRST

Requires human decision

- ? Adding new dependencies
- ? Changing database schema
- ? Modifying auth middleware
- ? Changing API response formats

This connects to Week 1's PRD Non-Goals. **Same principle: AI can't infer what you DON'T want.**

Before & After — Instruction Files in Action

⚠ WITHOUT INSTRUCTION FILE

```
claude "Add a new API endpoint  
for user profiles"
```

- ✗ Uses Express.js (you use Fastify)
- ✗ MySQL queries (you use Prisma)
- ✗ No error handling
- ✗ No tests generated

3-5 iterations to fix

✅ WITH INSTRUCTION FILE

```
claude "Add a new API endpoint  
for user profiles"
```

- ✓ Uses Fastify (matches your stack)
- ✓ Prisma ORM (follows your pattern)
- ✓ Zod validation (your rule)
- ✓ Error handling (your boundary)
- ✓ Vitest tests generated

1 iteration. Done.



The instruction file saved $3 \text{ iterations} \times 5 \text{ min} = \textbf{15 minutes per feature}$. Over a project, that's hours.

Part 3 — Git Workflows for AI Development

BAD — MIXED CONVERSATIONS

One conversation, multiple features:

- └ "Add login page"
- └ "Also add dark mode"
- └ "And fix that bug"

✗ Context polluted, AI confused

GOOD — FOCUSED CONVERSATIONS

One conversation per feature:

- Conv 1: Login → commit → done
- Conv 2: Dark mode → commit → done
- Conv 3: Bug fix → commit → done

✓ Clean context = better quality

BRANCH STRATEGY

main (protected – never push directly)

- └ feat/login-page
- └ feat/dark-mode
- └ fix/navigation-bug
- └ chore/update-deps

Each branch = one feature
= one conversation
= one atomic commit

Commit Message Convention

Clear history helps you AND the AI understand what changed

type(scope): short description

Longer description if needed.

```
git commit -m "feat: add search
```

```
- Search API with pagination  
- Tests: 3 new, all passing"
```

feat

New feature

feat: add search endpoint

fix

Bug fix

fix: prevent duplicate login

test

Adding tests

test: add auth edge cases

refactor

Code cleanup

refactor: extract validation

docs

Documentation

docs: update API readme

chore

Build/deps/CI

chore: upgrade vitest

Code Review Checklist for AI-Generated Code

Before committing ANY AI-generated code — the AI is always confident. Your job is to verify.

CHECK	WHY	COMMAND
Does it compile/run?	AI generates invalid syntax	<code>npm run build</code>
Do existing tests pass?	AI breaks existing features	<code>npm test</code>
Hardcoded values?	AI loves magic numbers	<code>grep "TODO FIXME"</code>
Error handling present?	AI often skips error cases	Review catch blocks
Types correct?	AI generates loose types	Check for "any"
Over-engineered?	AI adds unrequested features	Compare to plan
Security issues?	SQL injection, XSS, secrets	Review inputs/outputs

Lab Exercise — 30 Minutes

Apply the EPCV cycle to add ONE new feature to your Week 3 prototype



0-5 min

Setup

Create instruction file + feature branch



5-10 min

EXPLORE

Ask AI to explore your codebase



10-15 min

PLAN

Get AI plan (no code yet!)



15-25 min

CODE

Implement one step at a time



25-30 min

VERIFY

Run tests + review diff + commit

Submit: Instruction file + git log showing atomic commit + 100-word reflection

Key Takeaways

1 EXPLORE before you code — the AI doesn't know your codebase. YOU have to show it.

2 “Don't write code yet” is the most powerful AI instruction. Plans before code.

3 CODE incrementally — one step at a time, review each step before proceeding.

4 VERIFY before commit — tests + diff review + atomic commits.

5 Instruction files: write once, get consistent AI behavior in every conversation.

6 One feature per conversation — clean context = better code quality.